

프론트엔드 트랙 — 파운데이션·신뢰성 → 포트폴리오 캡스톤

이 트랙은 무엇인가. React 생태계의 신뢰성(실패 모드·데이터 패칭·상태 소유권·RSC 경계)을 손으로 쌓고, 그것을 공개 포트폴리오 사이트 portfolio-site (Next.js)로 담는다. 캡스톤이 포폴 그 자체라 효율적이다 — 배우면서 곧 취업용 페이지가 남는다. 코드 재현이 아니라 판단력. 언제 시작하나. 42 트랙에서 M0 규율 + pong-pong으로 TypeScript를 이미 익힌 뒤 분기해 들어온다. 그래서 TS 선행은 여기서 다시 다루지 않는다. (TS를 더 깊게 파고 싶으면 grounded-travel 이 TS 대장이지만 선택이다 — pong에서 TS를 했다면 건너뛰어도 된다.) 읽는 법. 위에서 아래로 한 프로젝트씩. 각 블록은 자족적이다 — 노트 순서·재구성 방법·L3·검증·합정·완료 바가 블록 안에 다 있다. 이 문서 하나로 트랙을 완주하도록 썼다. 딱 한 번만 외우는 공통 어휘 (이후 각 블록은 이 말만 쓴다): - L3 = RSC 경계·낙관적 업데이트 롤백·스테일 응답·에러 경계 등 실패 모드 결정. 답지 덮고 백지에서 코드+이유를 재구성하고 실패 모드까지 방어돼야 "끝". - L2 = 구조·흐름·책임 분리(펼쳐 설명되면 충분). L1 = 보일러플레이트·스타일·카탈로그 등록(읽고 통과). - 핵심 동작 = 이 트랙의 파운데이션·신뢰성은 triplet(2A) 리듬이다 — ① docs: ...contract (계약) → ② 덮고 계약만 보고 직접 구현 → ③ feat 답지 diff → ④ test 검증. 답지 끝 ## 기억/설명 Level 에 L3/L2/L1이 적혀 있으니 들어가기 전에 본다. 포폴은 순차(2B) 기능 커밋이라 곁이 다르다(아래 3번). - 커밋 규율 (학습 산출물을 커밋할 때 — 이 트랙 전 레포 공통): 제목은 영어 명령형 <type>(<scope>): ... (type ∈ feat·fix·docs·test·refactor·chore·build·ci·perf), 본문은 한국어 【근거】 왜 / 【변경】 무엇 / 【검증】 어떻게 (잡일·merge는 면제), AI 공동저자 트레일러 없음. 날짜는 KST-author==committer·근무시간 09:00-21:59·분초 랜덤·중복 금지. dual-form 문서는 답지 docs(commits) / 문제지 docs(practice) 스코프로 나눠 바뀐 파일만 add(-A 금지).	
전체 경로:	
frontend-foundations-training → frontend-reliability-training → ★portfolio-site(캡스톤) (④·2A·zustand) (⑦+⑥·2A·React 대장) (④·2B·Next.js 자족)	

1. frontend-foundations-training — React 기초 위밍업(Zustand)

무엇/왜. Next.js App Router 단일 흐런 앱 위에서 과제 20개(spine 12 + capstone 3 + bridge 5)를 반복한다 — semantic page → 컴포넌트/props/state → events/controlled forms → list/detail 라우팅 → local CRUD/storage → HTTP route handler → API CRUD fetching → loading/error/empty → Zod 폼 검증 → 수동 세션 로그인 → pagination/filtering → 파일 업로드. 신뢰성 본대(2번) 전에 곡선을 부드럽게 하는 위밍업이다. 이 트랙에서 얻는 것. App Router 서버/클라이언트 경계의 첫 감각 + 공유 타입 계약을 먼저 세우는 습관. 장르·리듬. 장르 ④ 따라만들기(exercise 카탈로그, 공용 ExerciseDemo 런타임)·triplet(2A): docs: frame → feat: implement → test: cover. 답지 73(비-merge) / 문제지 12. 노트 읽는 순서 (frontend-foundations-training/notes/). zustand (스택 교과서) + 짝 reference-impl/ (Zustand 미니 구현). 노트가 한 편뿐이라 가법다 — 핵심은 노트가 아니라 과제 카탈로그를 도는 것.	
재구성·커밋 활용법. 묶음으로 본다 — - 000~005 기반 — L3: 000(check:repo 검증 표면), 002(공유 데이터 계약 ExerciseCatalogItem · TrainingItem · ApiError · PageResponse<T> · SessionUser), 004(catalog helper + ExerciseDemo 공통 런타임·slug 기반 패널 선택). - 007~053 spine 12과제(위 목록을 frame→register→test로). - 055~065 branch capstones(board/commerce/task가 spine 결합). - 067~085 bridge — server/client 경계·cache·error fallback·접근성·낙관적 UI를 작게 한 번씩. L3: 067(route file / Client Component / serializable props가 나뉘는 책임 모델). - 087~094 project checks·런타임 안정화. L3: 092(query 정규화·page clamp·삭제 후 보장·localStorage 범위). L3 — 손으로 백지 재구성할 것. App Router 서버/클라이언트 경계(Provider 수명, "use client" 격리), 공유 타입 계약 선행 (PageResponse<T> · ApiError · SessionUser), catalog 단일 진실 모델(slug→라우트·검사·UI 자동 추론), pagination bounds·localStorage 안정화. 검증. make check:repo (lint → typecheck → test → check:docs). 합정. 카탈로그·범프류 반복 등록(catalog-bump)은 묶어서 빠르게 — 전부 같은 무게로 돌지 마라(세 커밋 사이클이 이미 자동화돼 있다). bridge 과제는 foundations 범위만(낙관적 업데이트도 즉시 피드백까지, 롤백·동시성은 2번에서). 실제 인증·영속 저장소 없음(fixture + memory store). 완료 바 → 다음으로. 서버/클라이언트 경계와 공유 타입 계약을 백지로 설명되면 — 이제 신뢰성 본대로. foundations는 위밍업이라 깊이를 욕심내지 말고, 진짜 L3는 다음 레포에 있다.	

2. frontend-reliability-training — React 대장(신뢰성 결정 패턴)

무엇/왜. pnpm 워크스페이스 흐런 앱 2개(react-main + next-sub)로 신뢰성 결정 패턴을 과제로 못 박는다 — error boundary, optimistic update rollback, virtualized list, offline cache, Suspense streaming, auth-protected UI, performance budget. 이 트랙의 대장(심화 봉우리 ★)이자 L3의 본대다. 이 트랙에서 얻는 것. "화면이 <i>틀리게</i> 동작하는 모든 경우"를 코드로 막는 판단력 — 포폴을 견고하게 만든다. 장르·리듬. 장르 ①(코드 재구성) + ⑥(test-first: 실패 예측·계약·테스트로 판단)·triplet(2A): docs: ...exercise contract → feat: implement → test: cover → route 연결. 답지 114(비-merge) / 문제지 60 — 대형. 그룹당 진짜 L3는 3~4개뿐(나머지 route-only 배선은 L2). L3만 깊게. 노트 읽는 순서 (frontend-reliability-training/notes/ , 묶음별). 루트 notes/ 는 평평하니 아래 묶음으로 읽는다(처음 보는 것만 1부부터): - React 코어: react → react-server-components → react-router-dom - 데이터 취득: tanstack-react-query → msw - 폼·검증: react-hook-form → hookform-resolvers - 테스트: vitest → testing-library → jsdom → jest-axe → storybook - 도구/스크립트: eslint → typescript-eslint → javascript-esm → nodejs-scripting → repo-guard-scripts 재구성·커밋 활용법. 구간으로 본다(번호는 docs/commits/README.md 의 phase 경계로 확인) — 000~007 저장소 기반 — L3: 001(pnpm workspace 단 일 명령 표면), 003(react-main 카탈로그/레지스트리), 005(next-sub App Router scaffold), 007(repository guard). - 009~055 React essential — L3: 009~011(zod 폼 검증 실패 분리), 016~018(async state·query key 경계), 022~024(local error boundary), 028~030(accessible modal focus 계약), 035~040(URL 기반 table state·control bounds), 042~048(optimistic update rollback·identity gate), 050~054 (URL locale source of truth). - 057~096 React advanced — L3: 057~062(virtualization 안전·scroll index 정밀), 064~069(cache freshness/stale fallback·MSW 실패 fixture), 071~076(design-system state priority·Storybook), 078~083(filtering URL/state 계약 정규화), 085~089(realtime ordering/dedup/reconnect), 091~096(performance budget·LCP/INP). - 098~129 Next.js — L3: 098~100(App Router file 경계), 107~110(server/client serialization· "use client" scope), 115~120(Suspense fast shell + slow section containment·streaming), 123~128(auth state matrix·민감 UI guard). - 131~136 통합·확장 (SRE/domain drill: route load·chaos payload·offline·virtualized large form). 각 L3 구간의 docs: contract 를 읽고 덮고 직접 구현 → test green → 답지 diff. L3 — 손으로 백지 재구성할 것. - RSC 경계 — "use client" scope, Server Component props serialization, 클라 state 격리. - 데이터 패칭 — query key collision, 스테일 데이터 렌더, mutation 낙관적 업데이트 vs 롤백, 동시성. - 상태 소유권 — URL을 단일 진실로(vs 컴포넌트 state), pagination/filtering bounds. - 에러 경계 — thrown promise capture, fallback UI addressability, Suspense 경계. - 그 외 — virtualization threshold(입력 크기 예산), realtime reconnect/dedup/ordering, 접근성(focus trap·aria·usual region). 검증. make typecheck / make test / make test-react / make test-next / pnpm check:repo . (Playwright·Lighthouse는 선택 smoke 층, 필수 게이트 아님.) 합정. 도구 체계 통합이 가장 큰 일(React 19/Vite vs Next 16/App Router 동시) — 햇갈리면 노트 묶음으로 돌아가라. blueprint/ (구현 지침)는 .gitignore 라 산출물 아님. 114커밋 중 route-only 연결 커밋이 다수 (L2) — 그룹당 L3 3~4개만 손으로, 나머지는 펼쳐 통과. 운영 KPI(실사용자 error rate·SLO)는 범위 밖. 완료 바 → 다음으로. RSC 경계·낙관적 롤백·스테일 fallback·에러 경계를 백지로 방어되면 — 이제 공개 포폴로. 여기서 익힌 실패 모드가 포폴의 견고함이 된다.	
--	--

3. ★ portfolio-site — 캡스톤: 공개 포트폴리오(Next.js 자족)

무엇/왜. Next.js + React 19 콘텐츠 주도 정적 포트폴리오 사이트. 캡스톤이 곧 취업용 공개 페이지다. 흥미로운 점: 이 레포는 자족 입문 대장이라 frontend-없이도 자체 docs/notes/systems/react.md (단독 45KB) 하나로 React를 처음부터 익힐 수 있다 — 그래서 신뢰성(2번)에 하드 의존하지 않는다(병렬성 참고). 이 트랙에서 얻는 것.* 콘텐츠/코드 경계가 깔끔한, 서버 컴포넌트만으로 정적 전개되는 진짜 배포물. 장르·리듬. 장르 ④ 따라만들기(콘텐츠 registry·README-first)·순차(2B) 기능 커밋: scaffold → 콘텐츠 계약 → 스타일 → 컴포넌트 → 페이지 → 콘텐츠 채우기 → 검증. 앞 두 레포의 triplet과 곁이 다르다 — 제목·본문 첫머리로 이벤 증분 목표를 잡고, 덮고 직전 상태에서 목표까지 구현 후 원본 대조. 답지 40 / 문제지 17. 노트 읽는 순서 (portfolio-site/notes/ + docs/notes/systems/). 구현 진입 전: nextjs → tailwindcss-v4 → css-tailwind-v4-directive → tsx-jsx · json · npm - 구현 중 펼침: postcss · simple-icons (tech 아이콘) · webpack - 자족 React 입문(frontend-* 생략 시): docs/notes/systems/react.md 한 편으로 컴포넌트·props·훅·렌더 모델·App Router· "use client" 까지. 신뢰성을 먼저 했다면 이 노트는 빠르게 훑고 지나가도 된다. 재구성·커밋 활용법. 묶음으로 본다 — - 000~002 기준점·scaffold(L2). - 004~006 콘텐츠 계약(L3) — src/content/*.json placeholder 구조 (site·profile·projects·skills·tech·stack·experience·journey·links·resume·contact) → src/lib/portfolio.ts 타입 계약 → getPortfolioContent() 조립 helper + 가시성(enabled:false)·source-only 정책. - 008~010 스타일(테마 토큰·모션·reduced motion, L2) · 012~015 컴포넌트(primitive·card·shell, L2). - 017~022 페이지(L3) — 홈 Design/Classic 듀얼 템플릿(같은 데이터 query 전환) · /projects · /projects/[projectId] 상세. - 024~034 콘텐츠·자산 채우기(42 커리큘럼 + reliability + full-stack 정렬). - 035~037 검증(L3) — helper unit(Vitest, hidden/source-only/query 계약 고정) → Playwright E2E smoke. - 038~040 reflection 페이지(/journey · /interview-map · /about 큐레이션 근거). L3 — 손으로 백지 재구성할 것. 콘텐츠/코드 경계(JSON registry vs 컴포넌트 로직), placeholder 계약 선행(파일·필드·구조를 먼저 고정), Design/Classic 듀얼 템플릿(같은 데이터로 query 전환), 서버 컴포넌트만으로 정적 전개(클라 state 없음), 콘텐츠 정책(enabled:false 숨김·source-only 상세 한정), helper→render→browser smoke 3층 검증. 검증. npm test (Vitest) / npm run test:e2e (Playwright) / npm run lint / npm run typecheck . 실제로 dev 서버를 띄워 두 템플릿·라우팅·viewport를 눈으로 확인한다. 합정. 콘텐츠 JSON이 한 registry에 모이므로 프로젝트 간 관계는 id 기반만. 외부 API cache·Lighthouse CI·visual regression은 후행 학습(범위 밖). 본문에 들어갈 프로젝트 집계(42 커리큘럼 + reliability + full-stack)에서 일부 항목은 enabled:false 로 보존만 한다. 완료 바 → 트랙 완주. 두 템플릿이 콘텐츠로 정적 전개되고 helper/E2E가 green이며, 콘텐츠 계약·듀얼 템플릿·source-only 정책을 백지로 방어되면 완주. 배포해 공개 URL을 커리어 산출물로 남긴다(스냅샷은 portfolio-public). 마지막으로 면접 대비로 프론트·클라이언트 신뢰성(스테일 응답·낙관적 업데이트·오프라인)을 백지 설명으로 담는다. 커밋 메모. 상단 <커밋 규율>대로 dual-form 스코프 분리(docs(commits) / docs(practice)), 변경 파일만 add. 콘텐츠 채우기 커밋과 구현 커밋의 phase 경계를 섞지 않는다.	
--	--

병렬성 — 이 트랙의 순서 규칙

- 신뢰성 ↔ 캡스톤은 병행 가능(백엔드와 다른 점). portfolio-site 는 자족 입문 대장이라 frontend-reliability-training 에 하드 의존이 없다 — 자체 systems/react.md 로 React를 처음부터 담을 수 있기 때문. 그래서 포폴 캡스톤을 신뢰성과 어느 정도 병행 학습해도 된다.
- 다만 권장 곡선은 신뢰성 먼저. 신뢰성에서 익히는 실패 모드(스테일·낙관적 롤백·에러/RSC 경계)가 포폴을 견고하게 만든다. 시간이 넉넉하면 순차(foundations → reliability → portfolio)가 가장 단단하다.
- 트랙 간(2 / 백 / 프론트): 서로 하드 의존이 없어(공통 게이트 M0뿐) 원리상 병렬도 되지만, 이 계획은 42를 먼저 완주하고 분기하는 단선이다. TS는 42의 pong에서 채워지므로 grounded-travel 은 선택이다.